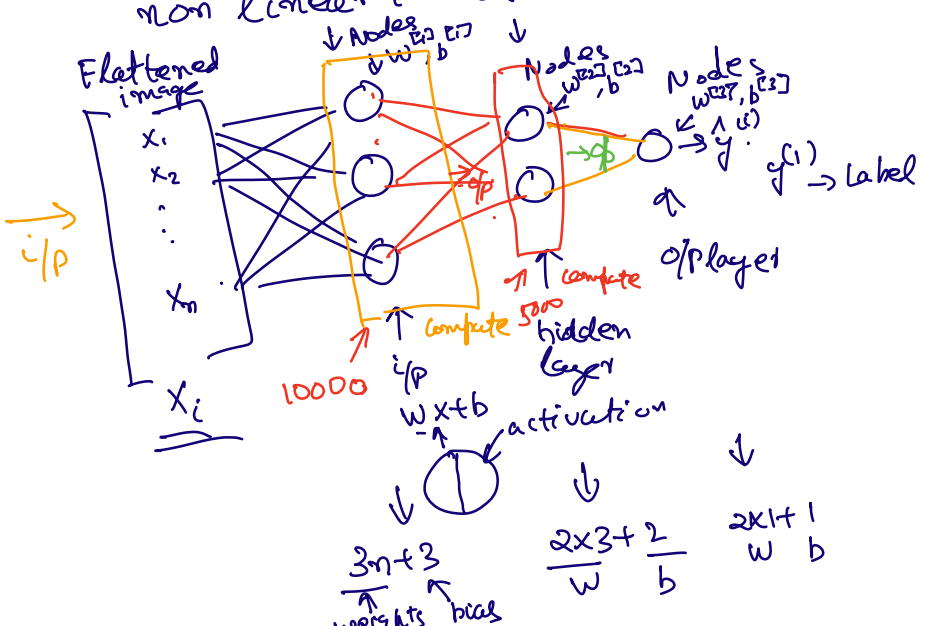
Neural Network Basics

1. Why neural networks?
2. How it works to optimise
   - forward propagation
   - back propagation
3. Significance activation functions
4. Pytorch Tensors, usage GPUs
5. Look at how to build a multi layer NN using Pytorch with GPU used for compute

Why NN?

- Most the interesting problems are non linear in nature



Layer 1 → Learning about edges
Layer 2 → Building edges and learning about shapes

First Layer

$$z^{[1]} = w^{[1]} x + b^{[1]} \leftarrow \text{vector of biases}$$

($w^{[1]}$: Params, x: matrix of example)

$$a^{[1]} = g(z^{[1]}) = \sigma(z^{[1]})$$

2nd Layer

$$z^{[2]} = w^{[2]} a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

3rd Layer

$$z^{[3]} = w^{[2]} a^{[2]} + b^{[2]}$$

$$a^{[3]} = \sigma(z^{[3]})$$

g is activation function
- sigmoid
- tanh
- ReLU
- LeakyReLU

Supervised Learning

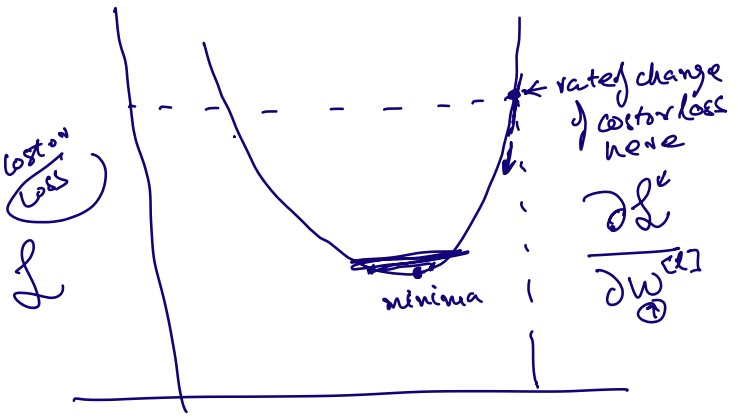
- How to minimise the error of prediction)

Loss function ($\mathcal{L}$)
Cost function ($\mathcal{J}$)

$$\mathcal{L}^{(i)} = -\left[y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log (1-\hat{y}^{(i)})\right]$$

- $\mathcal{L}^{(i)}$: minimise the loss
- $y^{(i)} \log \hat{y}^{(i)}$: Cross Entropy
- $(1-y^{(i)}) \log (1-\hat{y}^{(i)})$: Max Likelihood estimation

Back propogation → where we learn and comeup with new params which reduces cost or loss



$w^{[2]}$

$$w^{[2]} = w^{[2]} - \alpha \frac{\partial L}{\partial w^{[2]}}$$

α - Learning rate

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial g} \cdot \frac{\partial g}{\partial x}$$

$y = f(g)$

$g = f(x)$

$$\frac{\partial L}{\partial w^{[2]}} = \frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}}$$

$$\frac{\partial L}{\partial w^{[2]}} = \frac{1}{m} \sum_{i=1}^{m} \left(y^{(i)} - a^{[2]}\right) \cdot a^{[2]T} \quad -(1)$$

$$\frac{\partial L}{\partial w^{[2]}}, \frac{\partial L}{\partial w^{[1]}}$$

next set of forward p.p

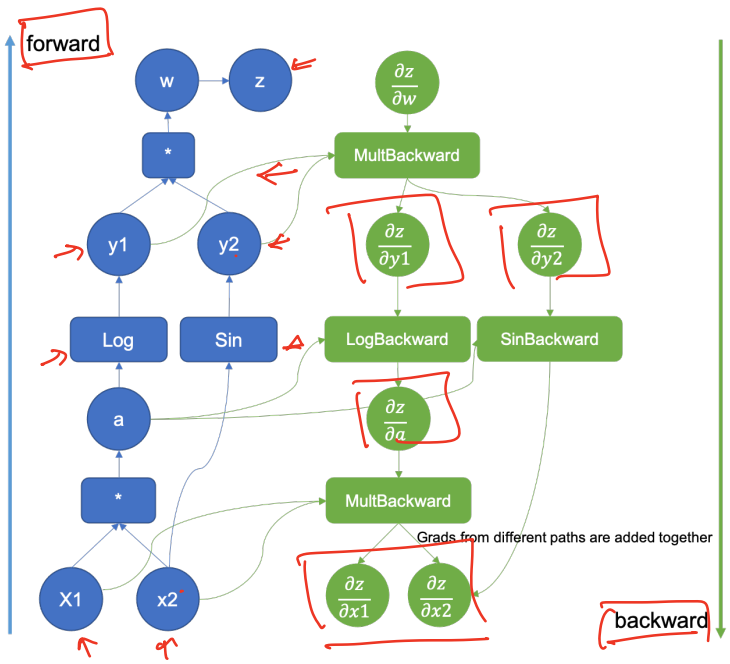
$$w^{[1]} = w^{[1]} - \alpha \frac{\partial L}{\partial w^{[2]}}$$

$$w^{[2]} = w^{[2]} - \alpha \frac{\partial L}{\partial w^{[2]}}$$

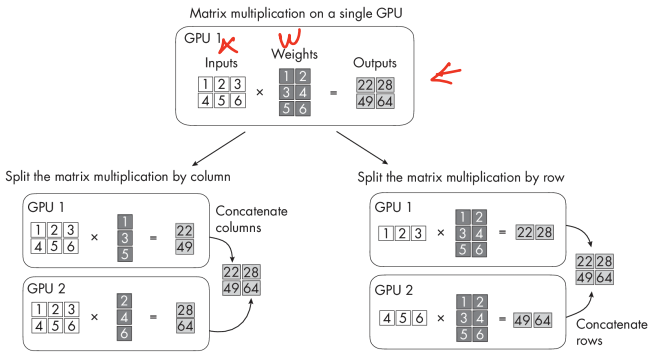
$$w^{[1]} = w^{[1]} - \alpha \frac{\partial L}{\partial w^{[1]}}$$

NN
USING
PYTORCH

computation graph with grads



Tensor parallel compute



Model parallelism

